Jackfruit Archive Binary Format Specification (.jack)

Binary Layout, Endianness, Metadata Dictionary & Bitstream Specification

UE24CS341A — Software Engineering | Release 1.0 (Sprint 1)

Jackfruit Archive Binary Format Specification (`.jack`)

Document Version: 1.0.0
Status: Approved Draft for Sprint 1
Lead Author: Saatwik (UC-01 Lead)
Co-Authors & Reviewers: Yuvraj (Archive Lead), Tanmayi (Testing/Decompression Lead), Vibhav (Integrity/Security Lead)

---

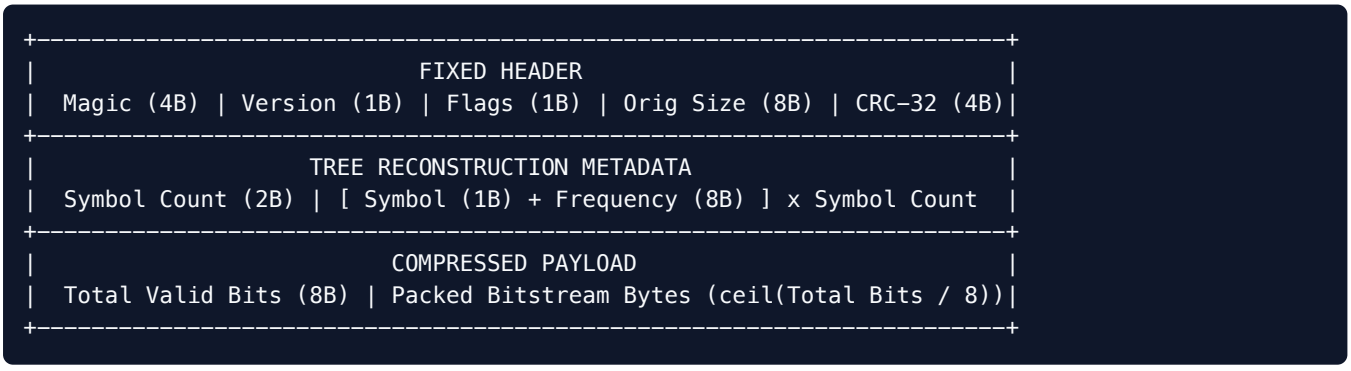
1. Overview & Objective

The `.jack` file format is a binary container format designed for lossless single-file compression using canonical Huffman coding. The format ensures: - **Portability:** Explicit endianness (Big-Endian / Network Byte Order) for all multi-byte integers. - **Self-Contained Decompression:** The archive header contains all necessary symbol metadata to reconstruct the exact Huffman decoding tree without external dictionaries. - **Strict Bit Precision:** A 64-bit valid bit-count prevents trailing byte-padding artifacts. - **End-to-End Integrity:** A 32-bit CRC checksum guarantees verification of decompressed data integrity.

---

2. Overall File Layout

A `.jack` archive consists of three contiguous sequential sections: 1. **Fixed Header (18 bytes):** Magic bytes, format version, flags, original file size, and CRC-32 checksum. 2. **Metadata & Tree Reconstruction Table (Variable size):** Symbol count and frequency dictionary. 3. **Payload Section (Variable size):** 64-bit total payload bit-count followed by the packed Huffman bitstream.



---

3. Detailed Byte-Level Layout

All integer fields greater than 1 byte are stored in **Big-Endian (Network Byte Order)**.

Section A: Fixed Header (18 Bytes)

Offset	Field Name	Type	Size	Description
`0x00`	`magic_bytes`	`char[4]`	4 Bytes	ASCII `"JACK"` (`0x4A` , `0x41` , `0x43` , `0x4B`). Identifies valid Jackfruit archives.
`0x04`	`format_version`	`uint8_t`	1 Byte	Format specification version. Currently `0x01` .
`0x05`	`flags`	`uint8_t`	1 Byte	Bitwise flags: • Bit 0: `IS_EMPTY` (1 if original file was 0 bytes, 0 otherwise). • Bit 1: `IS_UNCOMPRESSED` (fallback if compression expands file). • Bits 2–7: Reserved (must be `0`).
`0x06`	`original_size`	`uint64_t`	8 Bytes	Exact uncompressed file size in bytes (Big-Endian). Supports files up to $2^{64}-1$ bytes.
`0x0E`	`checksum_crc32`	`uint32_t`	4 Bytes	Standard IEEE 802.3 CRC-32 checksum of the original uncompressed file.

Section B: Tree Reconstruction Metadata

To guarantee deterministic Huffman tree reconstruction between Saatwik's encoder and Tanmayi's decoder, the archive stores a sparse frequency table of active symbols.

Offset	Field Name	Type	Size	Description
`0x12`	`symbol_count`	`uint16_t`	2 Bytes	Number of unique byte symbols present in the original input ($0 \le N \le 256$). If `0` , original file was empty.
`0x14`	`symbol_entries`	Array	$N \times 9$ Bytes	Array of N entries, each consisting of: • `symbol` (`uint8_t` , 1 Byte): The byte value (0 to 255). • `frequency` (`uint64_t` , 8 Bytes): Frequency count of this symbol in the original file.

Size of Section B: $2 + (N \times 9)$ bytes. Maximum overhead for all 256 ASCII/binary byte values is $2 + (256 \times 9) = 2,306$ bytes (~2.25 KB).

Section C: Compressed Bitstream Payload

Immediately follows Section B.

Offset	Field Name	Type	Size	Description
Variable	`total_bits`	`uint64_t`	8 Bytes	Exact number of valid encoded bits in the compressed payload stream.
Variable	`bitstream_data`	`uint8_t[]`	M Bytes	Encoded Huffman bitstream packed into bytes. $M = \lceil \text{total_bits} / 8 \rceil$.

---

4. Shared Bit-Ordering Convention (`BitWriter` & `BitReader`)

> **CRITICAL CONTRACT (Saatwik & Tanmayi):**
> All bits are written and read **MSB-First (Most Significant Bit First)**.

Packing Convention (`BitWriter` — Saatwik):

- Accumulate bits in an internal 8-bit buffer starting from the Most Significant Bit (Bit 7) down to the Least Significant Bit (Bit 0).
- Example: Writing bit sequence `1, 0, 1`:
- Step 1: Buffer starts at `00000000`, bit count = 0.
- Step 2: Push `1` $\rightarrow$ buffer holds `10000000` (bit count = 1).
- Step 3: Push `0` $\rightarrow$ buffer holds `10000000` (bit count = 2).
- Step 4: Push `1` $\rightarrow$ buffer holds `10100000` (bit count = 3).
- When 8 bits accumulate, the byte is written to the output stream, and the buffer is cleared.
- Upon calling `flush()`, if any bits remain ($1 \le \text{bits} \le 7$), the remaining lower bits in the byte are padded with `0`s, and the final byte is emitted.

Unpacking Convention (`BitReader` — Tanmayi):

- Read bytes sequentially from the stream.
- Extract bits from the current byte from Bit 7 down to Bit 0 using: $$\text{bit} = (\text{byte} \gg (7 - \text{bit_index})) \ \& \ 1$$
- Terminate reading exactly when `read_bits_count == total_bits`. **Do not read padding bits in the final byte.**

---

5. Edge Case Handling

1. **Empty File (0 Bytes):**
2. `original_size = 0`
3. `flags = 0x01` (`IS_EMPTY`)
4. `checksum_crc32 = 0x00000000`
5. `symbol_count = 0`
6. `total_bits = 0`
7. Total archive size: exactly 28 bytes.
8. **Single Unique Symbol (e.g. `"AAAAAA"`):**
9. `symbol_count = 1`.
10. By definition, a Huffman tree requires at least two nodes. For single-symbol files, assign the trivial code `0` of length 1 bit to the single symbol.
11. `total_bits = original_size`.
12. **All 256 Bytes Present:**
13. `symbol_count = 256`. Handled cleanly by `uint16_t`.

---

6. Validation & Integrity Requirements (Yuvraj & Vibhav)

When parsing a `.jack` archive, `ArchiveReader` and `IntegrityValidator` must enforce: 1. **Magic Bytes Check:** If the first 4 bytes $\ne \text{"JACK"}$, immediately abort with `ERROR_INVALID_MAGIC`. 2. **Version Check:** If `format_version > 1`, abort with `ERROR_UNSUPPORTED_VERSION`. 3. **Symbol Count Bounds:** If `symbol_count > 256`, abort with `ERROR_CORRUPT_HEADER`. 4. **Payload Length Check:** File size must equal $18 + 2 + (N \times 9) + 8 + \lceil \text{total_bits} / 8 \rceil$. If the physical file is smaller, abort with `ERROR_TRUNCATED_ARCHIVE`. 5. **CRC Check:** After decompression, compute the CRC-32 of the restored bytes. If calculated CRC $\ne$ `checksum_crc32`, report `CORRUPT_ARCHIVE_CHECKSUM_MISMATCH`.

---

7. Sign-off Matrix

Role	Member	Status	Notes
Architecture & Compression Lead	Saatwik	**Author / Approved**	Encoder & `BitWriter` designed against this spec.
Archive & Statistics Lead	Yuvraj	**Co-Author / Pending Review**	Serializer & parser logic mirrors this layout.
Testing & Decompression Lead	Tanmayi	**Approved**	Confirmed MSB-first `BitReader` agreement.
Security & Integrity Lead	Vibhav	**Pending Review**	Confirmed CRC-32 field position & bounds checks.